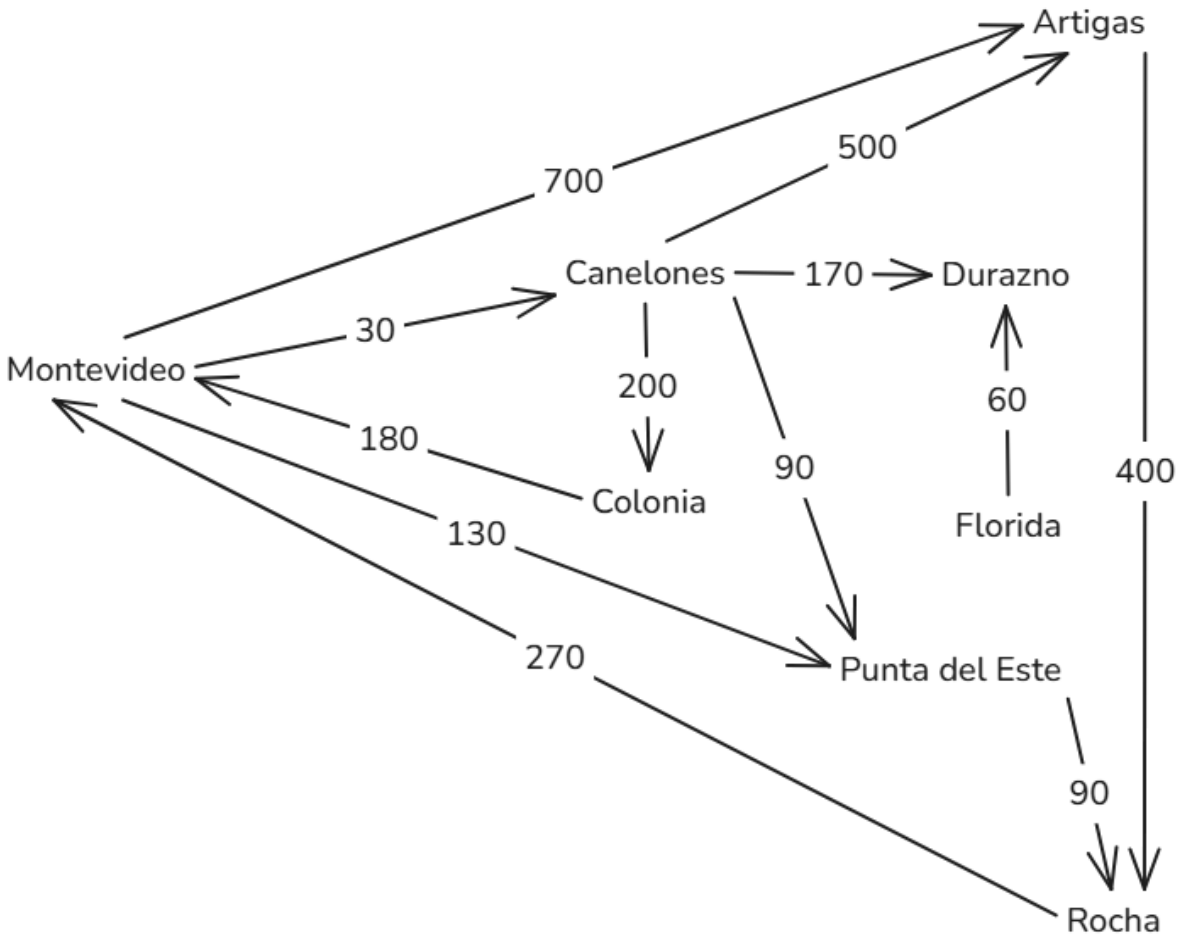
	Art	Can	Col	Dur	Flo	Mvd	PdE	Roc
Artigas	0	∞	∞	∞	∞	∞	∞	400
Canelones	500	0	200	170	∞	∞	90	∞
Colonia	∞	∞	0	∞	∞	180	∞	∞
Durazno	∞	∞	∞	0	∞	∞	∞	∞
Florida	∞	∞	∞	60	0	∞	∞	∞
Montevideo	700	30	∞	∞	∞	0	130	∞
Punta del Este	∞	∞	∞	∞	∞	∞	0	90
Rocha	∞	∞	∞	∞	∞	270	∞	0

Artigas → (Rocha, 400)
Canelones → (Artigas, 500), (Colonia, 200), (Durazno, 170), (Punta del Este, 90)
Colonia → (Montevideo, 180)
Durazno → (sin aristas salientes)
Florida → (Durazno, 60)
Montevideo → (Artigas, 700), (Canelones, 30), (Punta del Este, 130)
Punta del Este → (Rocha, 90)
Rocha → (Montevideo, 270)


Artigas
700
500
Canelones
170
Durazno
30
Montevideo
200
60
180
90
400
Colonia
Florida
130
270
Punta del Este
90
Rocha

Inte	S	W	Artigas	Canelones	Colonia	Durazno	Florida	Punta del Este	Rocha
–	M	–	700	30	∞	∞	∞	130	∞
1	M, C	Canelones	530	30	230	200	∞	120	∞
2	M, C, P	Punta del Este	530	30	230	200	∞	120	210
3	M, C, P, D	Durazno	530	30	230	200	∞	120	210
4	M, C, P, D, R	Rocha	530	30	230	200	∞	120	210
5	M, C, P, D, R, Co	Colonia	530	30	230	200	∞	120	210
6	M, C, P, D, R,	Artigas	530	30	230	200	∞	120	210

	Co , A								

Interfaz	Qué representa	Métodos principales
`IGraph<V, D>`	Grafo genérico: V es el tipo del vértice y D el dato de la arista	agregar, buscar y remover vértices; agregar, obtener y eliminar aristas; `vertices()`, `aristas()`, `adyacencias(v)`, `esConexo()`, `tieneCiclos()`
`IDirectedIGraph<V, D>`	Grafo dirigido; extiende IGraph	`successors(v)`, `predecessors(v)`, `gradoDeSalida(v)`, `gradoDeEntrada(v)`
`IDirectedGraphAlgorithms`	Algoritmos sobre	`dijkstra`, `floyd`, `warshall`, `obtenerCentroGrafo`,

	grafos dirigidos, separados de la estructura	`obtenerExcentricidad`, `obtenerTodosLosCaminos`, `recorridoEnProfundidad`, `recorridoEnAmplitud`, `calcularClasificacionTopologica`

Separar estructura y algoritmos permite cambiar la implementación del grafo sin tocar los algoritmos.

3.3 Colecciones para implementar `IDirectedIGraph`. Los métodos reciben vértices mediante un `Comparable`, por lo que ubicar un vértice exige recorrer el conjunto: O(V) en todas las opciones.

Estructura	Memoria	Agregar vértice	Agregar arista	¿Existe u→v?	Adyacentes de v
Matriz de costos	$O(V^2)$	$O(V^2)$ (redimensionar)	O(1)	O(1) con índices	O(V)
`Map<V, List<Edge>>`	O(V+E)	O(1)	O(grado)	O(V + grado)	O(V + grado)
`Map<V, Map<V, Edge>>`	O(V+E)	O(1)	O(1)	O(V)	O(V + grado)

Elegimos `LinkedHashMap<V, List<Edge<V, D>>>` (clase `DirectedGraph`):

Los grafos del problema son dispersos (8 vértices y 12 aristas frente a 64 celdas), así que la lista de adyacencias ahorra memoria.
Recorrer las aristas salientes, lo que más hacen Dijkstra, el DFS y la búsqueda de caminos, es directo sobre la lista.
`LinkedHashMap` conserva el orden de carga, de modo que los recorridos empiezan por el primer vértice representado.
Su costo es que `existeArista` y `predecessors` son más lentos que con una matriz; para estos tamaños no es relevante.